Züge in Usbekistan

Nachdem Usbekistan 2026 die Olympiade ausrichtet, braucht das Land dringend gute und schnelle Verbindungen zwischen den Städten. Die Regierung beschließt daher, ein Schienennetz zu bauen, um die Städte miteinander zu verbinden. Um das Land schnell und kostengünstig zu verbinden, sollen nach und nach n Verkehrsknotenpunkte gebaut werden. In einer ersten Phase werden die drei Städte Tashkent (1), Samarkand (2) und Bukhara (3) miteinander verbunden. Es wird jeweils eine Schiene von Tashkent nach Samarkand, von Samarkand nach Bukhara und von Bukhara nach Tashkent gebaut.

In einer zweiten Phase werden nach und nach $n - 3$ weitere Verkehrsknotenpunkte gebaut. Jeder neue Verkehrsknotenpunkt i ($4 \leq i \leq n$) wird mit genau zwei bereits bestehenden und benachbarten Verkehrsknotenpunkten a_i und b_i verbunden, sodass die drei Knotenpunkte $0 < a_i < i$, $0 < b_i < i$ und i ein neues Verkehrsdreieck bilden. Die Reise zwischen zwei **direkt** verbundenen Verkehrsknotenpunkten dauert genau eine Stunde.

Die Veranstalter der Olympiade wollen nun Ausflüge planen und müssen daher wissen, wie lange es dauert, um von einem Verkehrsknotenpunkt u zu einem anderen Verkehrsknotenpunkt v zu reisen. Hierzu fragen sie dich q-mal, wie lange es dauert, um von einem Verkehrsknotenpunkt u zu einem anderen Verkehrsknotenpunkt v zu reisen.

Implementierung

Deine Aufgabe ist es, die Prozeduren `init(n, edges)` und `calculate(u, v)` zu implementieren.

```
void init(int n, vector<pair<int,int>> edges)
```

Die Funktion wird einmal zum Anfang aufgerufen, um die Informationen über die Verkehrsknotenpunkte und die Verbindungen zwischen ihnen zu übergeben.

- Der Parameter n gibt die Anzahl der Verkehrsknotenpunkte an.
- Der Parameter *edges* ist ein Vektor von Paaren von Verkehrsknotenpunkten, die durch Schienen miteinander verbunden sind.

```
int calculate(int u, int v)
```

Die Funktion berechnet die kürzeste Distanz zwischen zwei Verkehrsknotenpunkten u und v.

- Der Parameter $0 < u \leq n$ gibt den Startverkehrsknotenpunkt an.
- Der Parameter $0 < v \leq n$ gibt den Zielverkehrsknotenpunkt an.
- Als Rückgabewert wird die kürzeste Distanz zwischen u und v erwartet.

Beispiele

Eingabe	Ausgabe	Anmerkungen
7 1 2 2 4 3 1 4 2 5 1 5 2 6 3 6 3 7 6 7	2 2 1 2 3	

Eingabe	Ausgabe	Anmerkungen
10 1 2 2 3 3 1 4 1 5 2 6 3 7 4 5 1 2 4 8 8 10 9 10 6 1	1 2 3 4 1	Entspricht Subtask 3

Subtasks

Allgemein gilt:

- $1 \leq a_i, b_i < i$
- $1 \leq u, v \leq n$
- Die Knotenpunkte a_i und b_i sind benachbart, d.h. es gibt eine Schiene zwischen a_i und b_i.
- $3 \leq n \leq 300\,000$
- $1 \leq q \leq 300\,000$

Subtask 1 (30 Punkte): $q \leq 100$

Subtask 2 (10 Punkte): $n \leq 5000$

Subtask 3 (20 Punkte): Für den Graphen gelten folgende Bedingungen:

- für alle $i > 6$ gilt $a_i \equiv i \equiv b_i \pmod 3$
- $a_4 = 1$, $a_5 = 2$, $a_6 = 3$
- $b_4 = 2$, $b_5 = 3$, $b_6 = 1$
- für alle Anfragen gilt $u_i \not\equiv v_i \pmod 3$

Subtask 4 (40 Punkte): Keine Einschränkungen

Limits

Zeitlimit: 3 s **Speicherlimit:** 1024 MB

Eingabe

Das Einlesen der Eingabe wird von einem von uns geschriebenen Programm, dem Grader, übernommen. In der ZIP-Datei zu dieser Aufgabe findest du für jede Programmiersprache eine Grader-Datei, die du zum Testen deiner Lösung mit deinem Programm kompilieren kannst.

Dieser Grader liest die Eingabe im folgenden Format ein: In der ersten Zeile steht die Zahl n. In den nächsten $n-3$ Zeilen stehen die Paare a_i und b_i der Verkehrsknotenpunkte, die den neuen Verkehrsknotenpunkt i verbinden. In der Zeile danach steht die Zahl q. In den nächsten q Zeilen stehen die Paare u und v der Verkehrsknotenpunkte, für die die kürzeste Distanz berechnet werden soll.

Eingabe	**Ausgabe**
n a_4 b_4 a_5 b_5 ... a_n b_n q u_1 v_1 u_2 v_2 ... u_q v_q	$dist(u_1, v_1)$ $dist(u_2, v_2)$ ... $dist(u_q, v_q)$